

ENC#YPTED: An Educational Game for Programming in the Unity Engine

Vincent Schiller

New Work Design Lab, Fachhochschule Dresden
v.schiller@fh-dresden.eu

ABSTRACT

In an era of digitalization, the relevance of programming in real-time development environments is constantly increasing. This also applies to the Unity Engine. Within the framework of a bachelor thesis, the first freely accessible Unity programming learning game called 'ENC#YPTED' was developed. ENC#YPTED aims to support a positive learning experience in the acquisition of programming skills. This is achieved by immersing the learner in a game world. The application was iteratively improved and refined according to user-centered development through user experience testing. This adaptation creates a high level of usability. Game duration and difficulty were not relevant for most of the test subjects to achieve their goals. Considering the high level of motivation and entertainment of the testers, it can be assumed that participants had a flow experience while playing ENC#YPTED. This paper describes the game concept and the user-centered design research accompanying the implementation process.

CCS CONCEPTS

• Human-centered computing; • Interaction design; • Interaction design process and methods; • User centered design; • Social and professional topics; • Professional topics; • Computing education; • Computing education programs; • Software engineering education; • Software and its engineering; • Software organization and properties; • Contextual software domains; • Virtual worlds software; • Interactive games;

KEYWORDS

Game-based Learning, GBL, Educational Games, Coding Games, User Experience Design, UXD, Unity, Game-Engines

ACM Reference Format:

Vincent Schiller. 2021. ENC#YPTED: An Educational Game for Programming in the Unity Engine. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 08–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3411763.3451852>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI '21 Extended Abstracts, May 08–13, 2021, Yokohama, Japan

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8095-9/21/05.

<https://doi.org/10.1145/3411763.3451852>

1 INTRODUCTION

Digitalization is advancing at an exponential rate. New digital formats are constantly emerging in a variety of industries. The software Unity is becoming increasingly relevant to these industries, as it is the world's "most widely used real-time 3D development platform" [1]. Additionally, Unity is driving the establishment of real-time 3D not only in the gaming world, but also in other areas such as architecture, automotive engineering, film, medicine, aerospace [1] [2]. It includes tools that can be used to develop classic 2D and 3D as well as interactive AR and VR experiences.

These tools aid in creating and designing particle effects, landscapes, and animations as well as simulated physical events. While Unity already provides a variety of components for classic tasks, such as in the areas of audio, physics and rendering, a lot of features still need to be programmed by the developer. Especially the core – the specific game logic – requires the creation of so-called scripts using the C# programming language.

There are many ways to acquire programming competencies and skills. In addition to search engines, forums, videos, books, and classes, games hold special potential in imparting knowledge in the field of computer science.

Digital Game-based learning which means the combination of educational content with computer games [3, p. 145] can facilitate the introduction to these programming topics. An important component of Game-based learning is gamification – the use of game elements in non-game contexts [4, p. 2] – as this can have a positive effect on both the effectiveness of learning and the motivation experienced [5]. Furthermore, games can eliminate uncertainties, can generate flow, and counteract negative learning experiences as e.g., failure as part of the programming process. Learning Environments are created which make it easier to experience the principle of cause and effect in a playful way.

Educational games exist for all common programming languages. However, no comprehensive educational game focusing on programming in the Unity Engine can be found on any of the common digital marketplaces [6, p. 20]. To change this phenomenon, a simulation game for learning Unity-specific programming paradigms called 'ENC#YPTED' was developed. This development followed a user-centered design approach. User-experience testing was used to explore how to design a digital game-based learning application with the learning objective to acquire programming skills in the Unity engine so that the experience is user-friendly, motivating, and extensible.

2 DESIGN RESEARCH

The chosen method of a user-centered design in the presented case consisted of the following phases: analysis, conception, implementation, and evaluation. The analysis mainly included research that

has partly been presented in the previous chapter 'Introduction' and the upcoming topics 'Literature Review' and 'Related Games'.

2.1 Related Work

ENC#YPTED aims to support user's acquisition of knowledge about programming in Unity. At the same time, a great user experience should be achieved. To follow both aspects during the development the following preliminary research was applied. This research explains how the brain reacts to external game influences [7, p. 220]:

- perception is subjective.
- memory decreases the more time passes.
- attention is limited to only a few resources.
- motivation influences our behavior decisively.
- emotions influence or are influenced by our cognitions, moreover they control our behavior.

In addition, existing game concepts were analyzed to derive corresponding insights for the own concept. The (coding) games 'SQL Island', 'Pony Island', 'SoloLearn', 'Swift Playgrounds' and 'CodinGame' were compared in the categories 'target audience', 'learning objectives', 'story', 'design', 'features', 'platform' and 'game duration', which lead to the following findings:

- SQL Island convinces mainly with its humorous story instead of elaborate graphical game worlds and thus shows the importance of a good storyline and the associated motivation to learn. An exciting story was therefore considered to be important for the own concept, especially since programming does not seem to be exciting for many at first.
- The game setting of the indie horror game Pony Island offers effective tension generation with simple means by combining visual effects with dramatic sounds. This was partly adapted to the own game concept.
- SoloLearn, however, manages to reach millions of users without any storytelling or graphical game elements. This can be attributed, among other things, to the platform's reward and comparison system. Certificates, the feeling of competition and a possibly better leaderboard ranking of the player are important motivating factors of this serious game. Since these approaches require a relatively large user base, they did not really have an impact on the implemented concept but are interesting for future work on the game.
- CodinGame offers especially experienced programmers a lot of room to experiment, improve and compete and is therefore also very community-oriented.
- In contrast, Swift Playgrounds relies on rewards in the form of entertaining animations of the to-be-controlled avatar. The application is characterized by the combination of the emotional connection to the game character with the playful testing of different solution approaches.

The idea of an entertaining, exploratory search for solutions plays a big role in many of the applications. As Jerome Bruner already speculated in 1961, "figuring out" or "discovering" approaches have a positive effect on learning [8]. Especially in the world of programming, which thrives on trial-and-error processes, this provides particularly interesting possibilities for developing competences by playing games.

2.2 User-Experience Tests

The crucial factor for the successful implementation of such a user-centered method is the early involvement of potential users in the development process. Regular evaluations and subsequent iterative improvements of the concept and implementation ensures the creation of a user-centered game experience instead of an idea defined by the developers [9].

During the implementation of ENC#YPTED, ten user experience tests were conducted with a total of 16 people. They took place at different stages of development and served to evaluate the previously added, modified, or enhanced features. The age of the respondents ranged from 19 to 61 (M: 26.75, SD: 10.59). The participants were observed while using the application and were asked about their user experience. When questions came up, they were answered directly, and the core problems were noted with the objective to find intuitive solutions for them over time. The duration and scope of the tests always varied according to the user-centered conversation approach.

For the documentation of the user experience tests (hereafter 'UX tests'), protocols were prepared in which both problems and corresponding solutions were recorded.

Based on the findings, the application was iteratively improved and further developed.

3 FINAL GAME CONCEPT

ENC#YPTED is a dystopian computer game with simulation-like approaches which is placed in the following game setting: At the beginning of the educational game, the user enters a Windows-like operating system, whose functionalities they first get to know in an explorative way. By using the game's own web browser, they are led to download a virus. After this executes itself, a blue screen occurs, whereupon the operating system reboots itself with a gloomy aesthetic. From this point on, players can no longer access the previously unknown data on the virtual desktop, as it has been encrypted. Only through the code-based collection of files in the game world, the users can unlock and open the files on the desktop again. They have to discover the solutions to these tasks by themselves, whereby their possibilities to act are limited. When the users have successfully solved all levels and thus saved all their data, the virus leaves the operating system again.

3.1 Learning Objectives

Users will learn Unity-specific methods as they progress through the game. The three levels each deal with specific topics, such as input, physics, and animation, in combination with basic C# use cases, such as the use of vectors, loops and logic. The learning goal is to identify connections and learn how to seek external help for various problems. It is not the intention of the application that users have a comprehensive knowledge of the Unity Engine after playing the educational game. With its graphical user interface, Unity integrates many more facets than programming which have not been relevant to this project. However, users should be able to easily acquire the rest of the knowledge about the engine in various ways.

3.2 Target Group

The game primarily aims at an enhancement of the learning or teaching experience of students and teachers in the field of

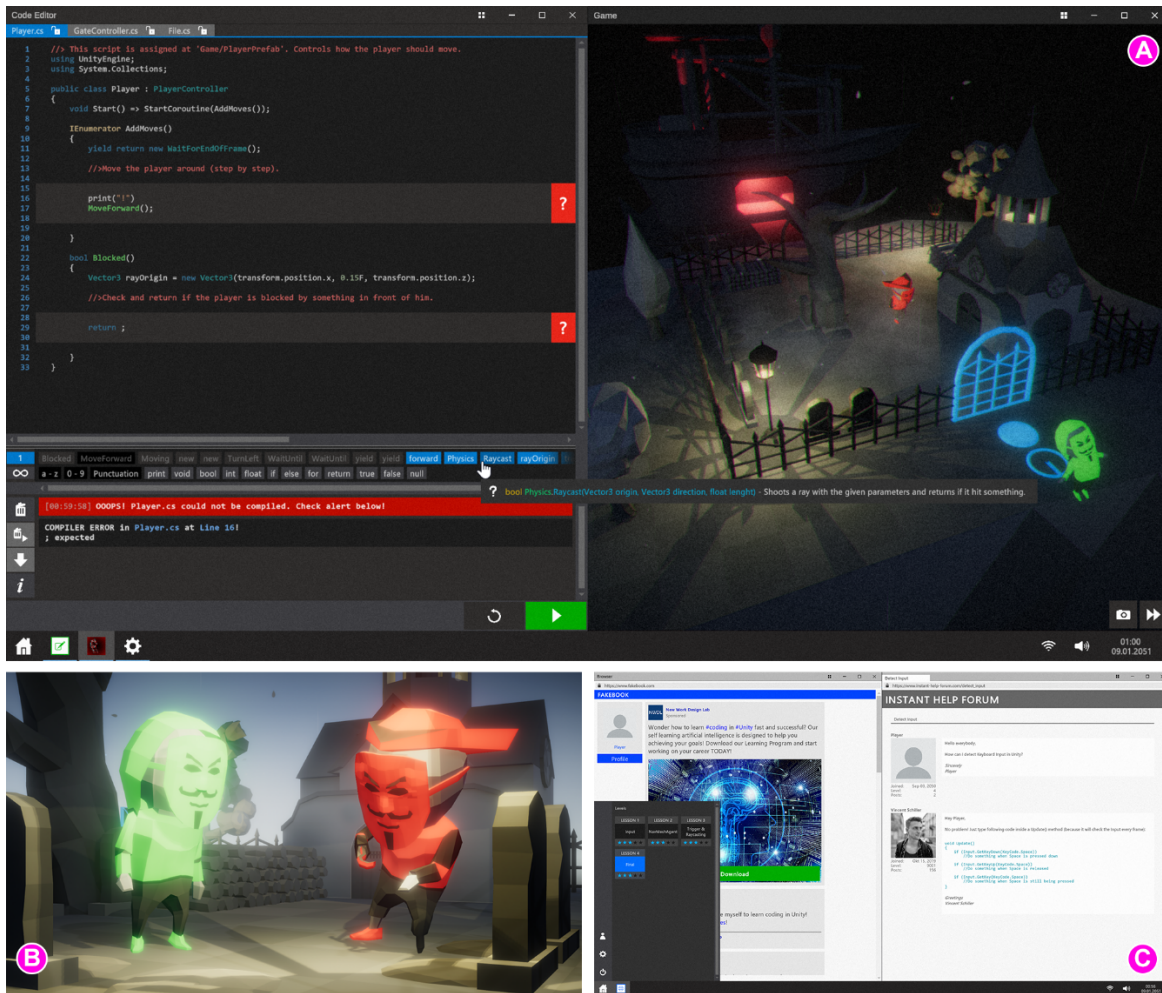


Figure 1: ENC#YPTD - A) User interface, B) Character design, C) Operating system design including browser with help forum.

computer science and game design, especially Unity Development. The knowledge of English and basic programming skills (ideally in C#) are prerequisites to achieve the learning objectives. Since the developed application is a prototype with three levels, the chosen topic is not yet comprehensively covered. Should further levels be developed in the future, which give a complete insight into the Unity-specific paradigms based on a clearly defined, common thread, there is the potential to use the application in teaching after a one-semester basic lecture on object-oriented programming. In the case of the inclusion of object-oriented fundamentals with the programming language C# within the application, even the use for students without prior knowledge is conceivable.

3.3 Design of the User Interface

The design concept is based on the simulation of a realistic-looking, abstract Windows operating system to enhance immersion. Application windows can be transformed arbitrarily, thus giving players the freedom to set a personalized screen layout. To manage the open windows and to make the simulation more realistic, a taskbar and a

start menu were also implemented (Figure 1 – C Left). A reduction to the most important graphical elements perfectly serves the goal of being able to control the attention of users during the game. The specially created code editor (Figure 1 – A Left) is aesthetically based on a standard Unity editor (Visual Studio) so that users will hardly have to change when switching from ENC#YPTD to the real Unity application due to the familiar design.

3.4 Main Features

The features include an implemented runtime compiler, which makes it possible to execute the code written by the player within the runtime of the application and, thus, instantly influence the game world (Figure 1 – A Right). This code is highlighted in different colors with syntax highlighting and compared with an allowlist before execution (Figure 2 - 4). Errors during execution are displayed in comprehensible console output (Figure 2 - 3). If at some point the player cannot find the solution without help, it is possible to use the 'Instant Help Forum' within the application to get immediate help on questions (Figure 1 – C Right). When the player completes

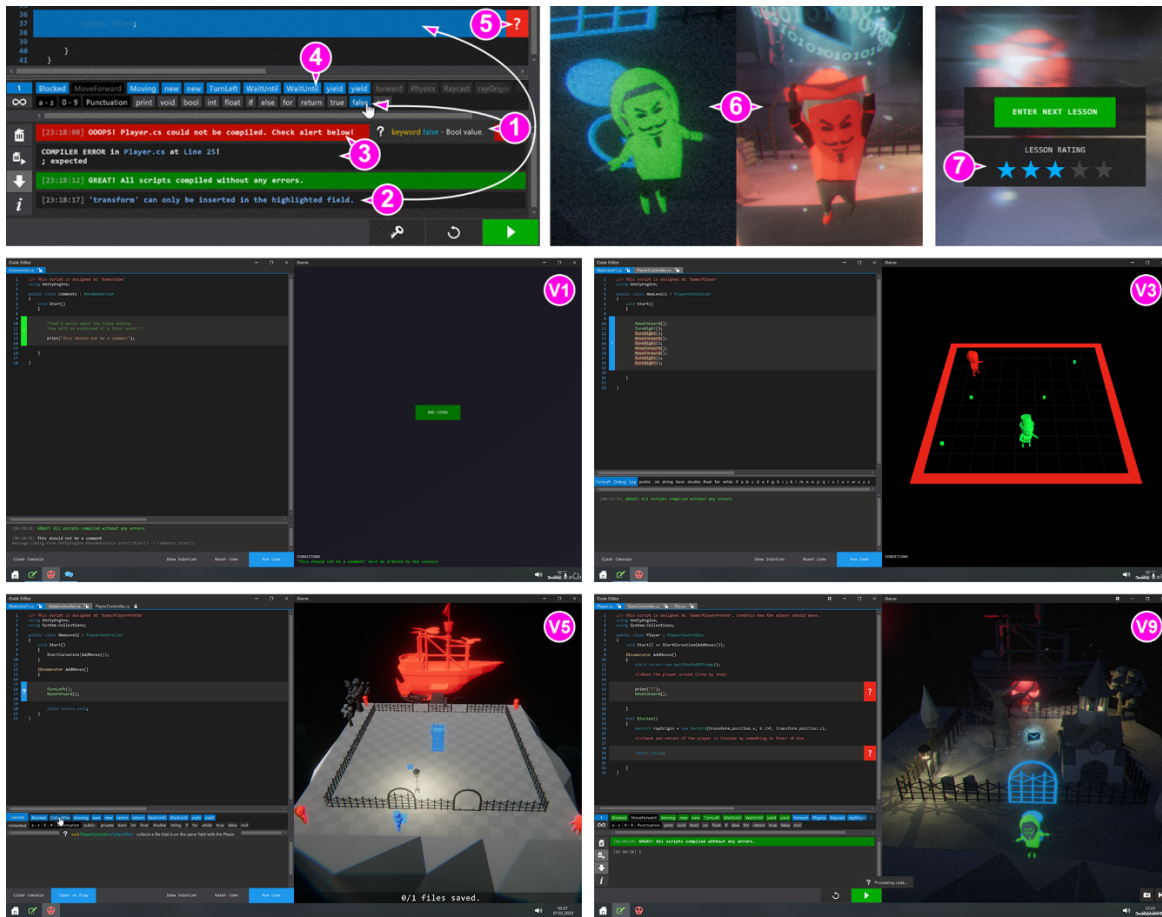


Figure 2: ENC#YPTED – 1-7) Usability and motivation features, V1/V3/V5/V9) increasing prototype versions.

a level, he has the option to view the decrypted files and play a new level which appears in the Windows-like start menu.

4 USER-CENTERED DESIGN PROCESS

4.1 Optimization of the Usability

The UX tests have shown that the usability is strongly dependent on the target group definition. In addition to that, it was attempted to use familiar functionalities and habits to a large extent, e.g., the Windows-like operating system or the structure and design of the code editor. Furthermore, it was done to enable users to get started with ENC#YPTED without a long learning phase. It also aims to increase immersion (experiencing the virtual environment as close to reality as possible). Through this simulation-like state, all important control elements and features were directly accessible and personalized through the free arrangement of windows.

The test also revealed user expectations based on previous experience and habits. Especially, unfulfilled expectations had an immediate negative impact on the user experience. One example for this is the undo/redo functionality in the code editor, which has been missing up to now.

In addition to intuitive use, it was important to increase the stimulus intensity in accordance with the increasing expertise of the users. For example, an expert mode was introduced that displays more information to experienced users than to users with little prior knowledge, to avoid overtraining of the users' processing capacities. Much emphasis was also put on the precise description of unknown stimuli to avoid being misunderstood. The introduction of tooltips was an essential part of this (Figure 2 - 1). Thus, many texts were replaced by icons with tooltips, which made the user interface much clearer. To support users' attention processes in a way that is more conducive to learning, several indicators were used to draw attention to a possible interaction, such as color, cursor, and size changes (Figure 2 - 5)

In some cases, users also attempted to perform forbidden or impossible actions. Without feedback on why the intended action did not work, subjects suddenly felt helpless or overwhelmed. For this reason, hints in the console and visual signals in the form of flashing boxes (Figure 2 - 2) were implemented for typical user errors. If the users now try to execute code that cannot be compiled, a clear and concise error message is displayed (Figure 2 - 3). The design of the error message follows the principle of showing possibilities for improvement instead of marking errors, thus avoiding

frustration, and maintaining the motivation to learn. Error sources had to remain manageable and the actual operating system of the users had to remain secure. This was one of the reasons why the allow-list (Figure 2 - 4) was developed, as otherwise crashes and data loss could be the undesired result.

4.2 Optimization of the User's Motivation

An important basis of motivation is the emotional bond between the player and his/her character. All respondents of the UX tests enthusiastically stated that they found the application to be an exciting simulation due to its dystopian story. The motivation arising from the story, wanting to save their 'own' data, is simple, but at the same time very comprehensible for every computer user. This supports an identification with the protagonist and the game. This is strengthened by personalization, such as the definition of one's own name and avatar image, which were subsequently used for personal addresses of the player.

To make this identification even more immersive, two game characters were developed. One of them is a (data-) pirate and the other one embodied the player (Figure 1 – B). This gives the psychological conflict between AI and user another, physical level. Similarly, the collection of the files in the game world and the resulting unlocking of the data on the virtual desktop of the player have a similar effect. This immersive representation of the conflict allowed for the implementation of other features relevant to motivation, most notably animation and aesthetics. As one of the test subjects described, "the magic comes from the character." Unexpected game outcomes and funny animations (Figure 2 - 6) make the teaching and learning environment more fun for the users. The motivational benefit in the described phenomenon is the exploratory or accidental discovery of these game outcomes. Some of them represent intentional as well as accidental rewards through the direct influence of the person playing the game, and thus have a high motivational potential. All subjects reported eagerly anticipating the impact of their new code input.

Unexpectedly, the difficulty was hardly relevant for the subjects' motivation. Most of them could only test one, according to their own statements, exceedingly difficult level. Without assistance, the solution would have been (almost) impossible for some due to lack of prior knowledge. Nevertheless, all subjects stated that they had been consistently motivated. When the developer asked for the reason, the majority replied that they had been "gripped by ambition". The level tested by most of them was partially played for two hours – which is quite a long time, considering that the comparable game-based learning games of chapter 2 take just a few seconds or minutes to solve a level. According to the testers, the motivation in the developed application is constantly maintained by small milestones, which the players work out for themselves. The test subjects partly lost their sense of time and were surprised by the duration of the past playing time. This and the statements about the constant motivation in the present application indicate that a flow experience [10] can be triggered by ENC#YPTED and this despite high perceived difficulty. Due to the long playing-time of a level, which deals

with relatively few Unity-specific methods, it can be assumed that the knowledge transfer is significantly slower than with conventional methods, such as following YouTube tutorials. Furthermore, due to the high motivation potential and the entertainment factor, a faster overcoming of the entry hurdle is also conceivable.

Another motivating factor is the planned assessment of performance at the end of a level (Figure 2 - 7), which encourages the player to repeat it. This way, what has been learned could be internalized in a more elaborate way.

5 CONCLUSION AND FUTURE WORK

With 'ENC#YPTED', the first freely accessible programming learning game for the Unity Engine was developed during a Bachelor thesis. The, at this stage, unique and innovative prototype was designed user-centered, which proved to be highly effective.

It was found that the learning game has a high motivational potential, as the subjects tested a level for over two hours in some cases, despite the difficulty being too high. Nevertheless, the testers stated that they were always motivated and entertained, which is why a flow experience can be assumed. As has been shown, the knowledge transfer of the prototype is rather slow due to the extensive playing time with comparatively few topics covered. This balance between entertainment and knowledge transfer shows one of the biggest challenges in the implementation of such an educational game. However, due to the high motivation potential, it can be assumed that the entry hurdle is easier to overcome for programming beginners.

Future research is needed to determine and possibly optimize the actual learning success of the application, including a broader use of the system in the context of standardized UX tests on a large sample. To accomplish this, additional game levels and improvements are already in the planning, which will soon be used and evaluated by students in the Game Design lecture for Unity Development at the University of Applied Sciences Dresden (FHD).

Although the findings obtained relate to the Unity platform, they can also be partially adapted for other engines.

REFERENCES

- [1] Unity Technologies. (n.d.). Public Relations. Retrieved January 2, 2021, from: <https://unity3d.com/public-relations>
- [2] Unity Technologies. (n.d.). Real-time solutions, endless opportunities. Retrieved January 2, 2021, from: <https://unity.com/solutions>
- [3] M. Prensky. 2007. Digital Game-Based Learning. Paragon House Publ.
- [4] S. Deterding *et al.* 2011. Gamification: Toward a definition. In CHI 2011 Gamification Workshop Proceedings.
- [5] J. Hamari *et al.* 2014. Does Gamification Work? A Literature Review of Empirical Studies on Gamification. In IEEE 47th Hawaii International Conference on System Sciences. Hawaii.
- [6] V. Schiller. 2020. Digital Game Based Learning am Beispiel der Programmierung in der Unity Engine: Entwicklung eines Simulationsspiels für das Erlernen der Unity-spezifischen Programmierparadigmen. Retrieved January 3, 2021, from: <https://nbn-resolving.org/urn:nbn:de:bsz:14-qucosa2-705979>.
- [7] C. Hodent. 2017. The Gamer's Brain: How Neuroscience and UX Can Impact Video Game Design. CRC Press.
- [8] J. S. Bruner. 1961. The act of discovery. Harvard educational review.
- [9] C. Abras *et al.* 2004. User-Centered Design. In Encyclopedia of Human-Computer Interaction. Sage Publications.
- [10] M. Csikszentmihalyi. 1988. The flow experience and its significance for human psychology. In M. & I. S. Csikszentmihalyi (Eds.)